EVOKE: Reversible KV-Cache Eviction as Agent Working Memory

Anish Shrestha
Independent Researcher
siranishshrestha@gmail.com
github.com/Anyesh/EVOKE

Abstract

A long-running agent accumulates context (files read, tool output, search results) faster than a fixed KV budget can hold, and both standard responses lose something: truncation discards content the agent may need again, while re-prefill re-encodes it at full forward-pass cost every time the agent refers back. EVOKE (EVict and recOver KV cache Entries) makes eviction reversible. An evicted block's K and V tensors move to host RAM; when the agent needs the block again, a recompute-free splice writes the saved tensors back into the live cache with a single RoPE re-anchoring rotation. Agent context arrives as identifiable blocks (a file read, a tool result), so recovery is keyed by block identity: the spliced bytes are the tensors the model first computed, never re-encoded text. We add the required KV-paging primitives to a llama.cpp fork, build an eviction and recovery policy on them, and evaluate four model families on a single 16 GB consumer GPU. A recovered block is as usable for recall as a fresh re-decode (100 % vs 100 %; the evicted-not-restored floor is 0–5 %), at 5.9–7.5$\times$ lower lifecycle cost than re-prefill on Qwen 2.5 7B. Across three recall benchmarks, every recovery-less baseline (recency, StreamingLLM, H2O (Zhang et al., 2023), SnapKV (Li et al., 2024)) fails once the probed content has been evicted, while every recovery-bearing policy passes; the divide repeats at the one budget swept (b=1024) on hybrid Mamba/Attention and MoE-with-thinking models. Against a same-substrate InfLLM (Xiao et al., 2024b) adaptation the two schedulers trade places along the budget axis: InfLLM separates at b=512, the two tie at b=1024, and EVOKE leads at b=2048 with overlapping intervals. This revision also adds a forward-looking eviction signal outside the attention-history family: a ridge probe distilled from the Jacobian-lens workspace readout of Gurnee et al. (2026) scores blocks by content at prefill time, ranks the planted fact above SnapKV's and H2O's choices offline on both models it was run on (fact AUC 0.891 vs. 0.622 on Qwen 2.5 7B; 0.865 vs. 0.563 replicated on Qwen3-8B), and keeps it resident at every swept budget with no recovery path and no measurable decode overhead. We also measure where a recovered block should land: tail re-anchoring helps only near a long-context model's far edge, is neutral at normal distances, and hurts short-context models. Recompute-free recall of evicted KV follows ArkVale (Chen et al., 2024); EVOKE contributes the agent-session regime, identity-keyed addressing, primitives spanning pure-attention, hybrid, and MoE architectures, and the placement measurement.

1 Introduction

A long-running LLM agent accumulates context across a session: files it reads, tool output, search results, prior reasoning. When that working set outgrows the KV budget, the harness has two conventional moves, and both lose. Truncating the oldest history (the StreamingLLM pattern)

Preprint. Under review.

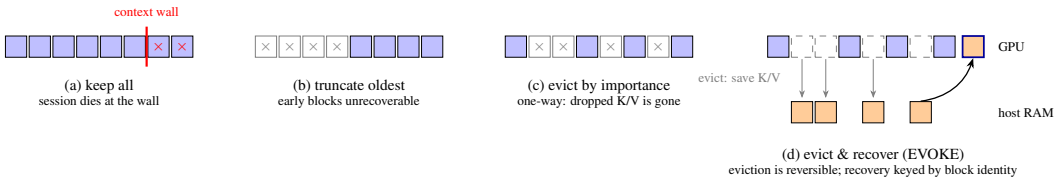

Figure 1: Four ways to handle an agent working set that outgrows the KV budget. (a) Keeping everything hits the context wall and the session cannot continue. (b) Truncation and (c) importance-based eviction (H2O, SnapKV) both treat eviction as one-way: once a block's K/V is dropped, a later question about it cannot be answered. (d) EVOKE moves evicted K/V to host RAM and splices the exact saved tensors back into the live cache when the agent refers to the block again, with no forward pass.

discards information the agent may need again. Re-prefilling the conversation on every step (the stateless chat-completions default, even with prefix caching) pays a full forward pass over old tokens whether or not they turn out to matter. Neither has a mechanism to bring back evicted content once the agent refers to it again.

EVOKE makes eviction reversible (Figure 1d). Cold blocks leave the GPU cache but keep their K and V tensors in host RAM; when a later turn needs an evicted block, a recompute-free splice writes the saved tensors back into the active cache through a single RoPE re-anchoring rotation. The KV cache becomes a working set over a two-level memory, and an eviction that turns out to be wrong is a latency cost rather than a permanent loss.

The reason recovery can be exact is that agent context is not an undifferentiated token stream. It arrives as discrete, nameable blocks: a file read, a tool result, a user turn. The harness that placed `file:config.py#0` into context knows when the agent asks about `config.py` again. EVOKE therefore keys recovery by block identity: selection of *which* evicted block to bring back may use similarity scoring, but substitution returns the exact bytes the model computed at first decode, never a re-encoding. This is the line that separates EVOKE from retrieval-augmented generation, and Section 4.2 draws it precisely.

EVOKE is not the first system to recall evicted KV without recompute. ArkVale (Chen et al., 2024) backs up evicted pages to host memory and recalls them per decoding step by an attention-importance estimate, at their original positions, in a per-layer paged cache. EVOKE differs in regime (a multi-turn agent session rather than one long prompt), in addressing (block identity rather than importance estimation), in placement (recovered blocks re-anchor to a live tail position, a choice Section 5.5 measures rather than assumes), and in reach (the primitives extend to hybrid Mamba/Attention, mrope, and MoE models on consumer hardware). We make no claim of advantage over ArkVale's full method (Section 7).

This paper contributes:

1. **Reversible-eviction primitives in a llama.cpp fork (Section 4).** `llama_kv_block_save` and `llama_kv_block_load` serialize a position range's K/V tensors to a host buffer and splice them back with per-cell RoPE re-anchoring; `llama_attn_capture_*` taps per-head attention weights for eviction scoring. The same mechanism covers pure attention, hybrid Mamba/Attention with mrope, iSWA dual caches, and MoE with thinking, all on a single 16 GB consumer GPU.
2. **An identity-keyed recovery policy and serving layer (Section 4.2).** Blocks enter the cache named by the harness; three pluggable recovery backends (discard, breadcrumb, `kv_restore`) make the recovery axis itself ablatable; an OpenAI-compatible server holds per-session KV across turns.
3. **A measurement of when reversible eviction matters and where recovered blocks should land (Section 5).** StreamingLLM, H2O, SnapKV, InfLLM, and an ArkVale-style recall arm are reimplemented on the same substrate, so the comparisons isolate policy from engine. Recovery-bearing policies pass recall benchmarks that every recovery-less policy fails; a 15-seed sweep against InfLLM shows a budget crossover rather than a uniform winner; and

a placement study bounded by never-evicted controls shows tail re-anchoring helps only near the context edge and is lossy relative to decoding the same content natively at the tail.

4. **A forward-looking, content-based eviction signal distilled from the model's workspace (Section 5.4).** Attention-history selectors can only rank blocks the model has already attended to. We distill the Jacobian-lens workspace readout of Gurnee et al. (2026) into a per-layer ridge probe (one dot product per position over residuals the fork already captures), validate it against attention oracles on pre-registered gates that replicate across Qwen 2.5 7B and Qwen3-8B, and show it selecting eviction victims better than H2O and SnapKV on the agent bench at every swept budget, before any decode history exists and at $-0.6\,\%$ measured decode overhead.

2 Background and Motivation

The budget is real. On the deployment class EVOKE targets (one consumer GPU), the KV budget is not an artificial constraint. A Qwen 2.5 7B at Q4_K_M takes $\sim$5 GB of a 16 GB card for weights, leaving $\sim$11 GB for activations and KV; at $\sim$60 KB of K/V per token (Llama 3.1 8B: $\sim$131 KB), an agent session that reads a few dozen files claims gigabytes of cache. Long-context models tolerate long inputs; they do not make holding everything free.

The two incumbent responses. Truncation is cheap and terminal: content past the window is gone, and Section 5.2 shows every truncation-shaped policy failing recall probes categorically. Re-prefill preserves information but pays $O(\text{tokens} \times \text{model FLOPs})$ on every reference: re-encoding a 1280-token block through Qwen 2.5 7B costs 232 ms on our eval host, paid again on each recurrence (Table 1). Prefix caching removes the cost only for unchanged prefixes; an agent that interleaves new turns with old references breaks the prefix continually.

What a third option requires. Reversible eviction needs three things to hold. The evicted tensors must splice back into a live cache that has since shifted, decoded new tokens, and evicted other blocks, and still be usable. The splice must be much cheaper than re-encoding, or re-prefill wins by simplicity. And something must know *which* bytes to bring back, without re-running the model over evicted text. Section 3 establishes each in turn.

3 Observations

3.1 Agent context is identity-addressable

Serving-side eviction systems must guess what matters: H2O and SnapKV score attention weights, ArkVale estimates page importance from key digests, all because a raw token stream carries no labels. An agent harness is different. Context enters as discrete artifacts with stable names (`file:config.py#0`, `tool:search#3`, `user#42`), and the harness sees the moment the agent refers back to one. Recovery in this regime does not need importance estimation; it needs an exact mapping from a name to saved bytes. The failure modes of recovery-less policies make the need concrete: asked about an evicted config value, the model answers "The maximum retry limit set in `config.py` is not explicitly mentioned" or "I cannot answer this question without inspecting the `config.py` file" (Section 5.2). The information was in cache once; the system simply has no way to bring it back.

3.2 Saved K/V splices back usable, even into a churned cache

Reversibility is only meaningful if a restored block works. We test this directly with a controlled greedy probe (`scripts/recovery_usability.py`, `scripts/churn_fidelity.py`): plant an unguessable value, evict its block, restore it, ask for the value. Figure 2 reports the result on a pure-attention model (Qwen 3-14B) and a hybrid Mamba/Attention model (Qwen 3.5 9B), with a single eviction and under heavy churn (the target block shifted by $\sim$128 compact-eviction position updates before being evicted and restored itself).

Recovered recall equals re-decode recall (both 100 %) on both architectures, with and without churn, while the discard floor sits at 0–5 %. A byte-level check (`scripts/verify_kv_fidelity.py`)

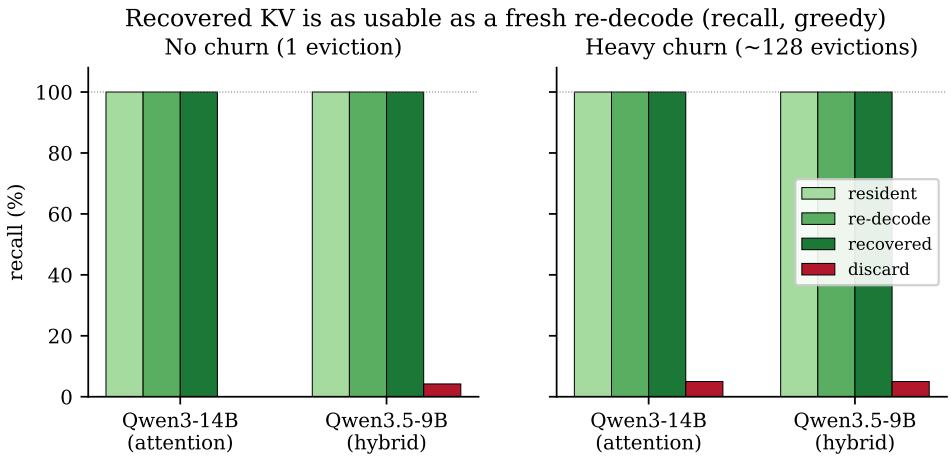

Figure 2: Recall from a recovered block matches a fresh re-decode and a never-evicted resident copy (greedy, value-in-answer), with a single eviction (left) and under ∼128 evictions of churn (right), on a pure-attention and a hybrid model. The `discard` floor (evicted, not restored) at 0–5 % confirms the probe is sensitive: the value is unrecoverable without the saved KV.

	Qwen 2.5 7B Q4_K_M				**Llama 3.1 8B Q4_K_M**			
n_tok	save	load	re-prefill	speedup	save	load	re-prefill	speedup
20	1.10	0.48	11.90	25×	2.65	1.78	12.58	7.1×
40	1.61	0.70	13.78	20×	3.82	1.94	14.62	7.5×
80	2.76	0.89	19.00	21×	5.82	2.46	19.94	8.1×
160	4.69	1.50	32.60	22×	10.30	3.60	32.55	9.0×
320	8.40	2.41	59.72	25×	19.82	5.77	61.23	10.6×
640	16.37	4.34	118.36	27×	39.05	8.87	121.83	13.7×
1280	31.90	7.25	232.18	32×	74.35	15.66	236.89	15.1×

Table 1: `kv_block_save`, `kv_block_load`, and equivalent re-prefill times in milliseconds across block sizes. Speedup is re-prefill divided by load (the latency paid at recovery). Save is paid once at eviction; the full evict-then-recover lifecycle (save + load) is 5.9–7.5× cheaper than re-prefill on Qwen and 2.6–2.8× on Llama, whose larger per-token K/V footprint (∼131 KB vs ∼60) raises transfer cost.

confirms the in-place restore reproduces a never-disturbed reference token-for-token (48/48 greedy continuations match). The splice returns usable context, not merely attendable bytes.

3.3 The splice is an order of magnitude cheaper than re-encoding

Table 1 profiles the primitive against re-prefilling the same tokens (`scripts/profile_recover.py`, block sizes 20 to 1280 tokens, Flash Attention on). Re-prefill scales with model FLOPs; the splice scales with bytes moved.

The gap widens with block size: at 1280 tokens the load is 32× faster than re-prefill on Qwen. Throughout this paper "recompute-free" is shorthand for "no model forward pass," not literal zero compute; these numbers measure end-to-end load including the re-anchoring rotation.

Together the three observations specify the design: name blocks at ingestion (Section 3.1), save K/V at eviction and splice on demand (Section 3.2), and let the splice's cost advantage (Section 3.3) absorb occasional wrong evictions.

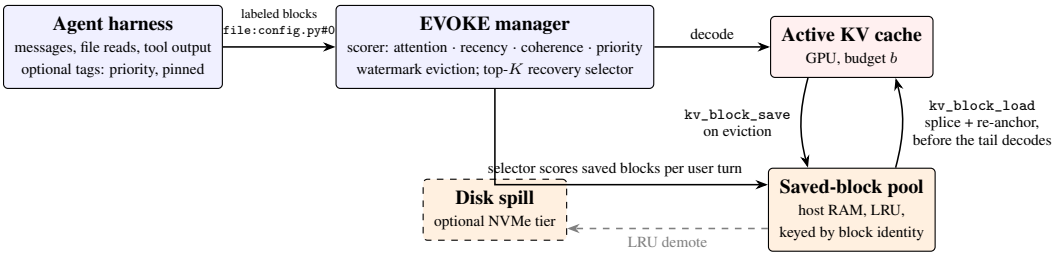

Figure 3: EVOKE design overview. Blocks arrive from the harness already named and decode into the budget-bounded active cache. When the active set crosses the high watermark, the scorer picks the lowest-value blocks and `kv_block_save` moves their K/V to the host pool. On each user turn, the recovery selector scores saved blocks against the incoming message and `kv_block_load` splices the chosen blocks back, recompute-free, before the new tail decodes.

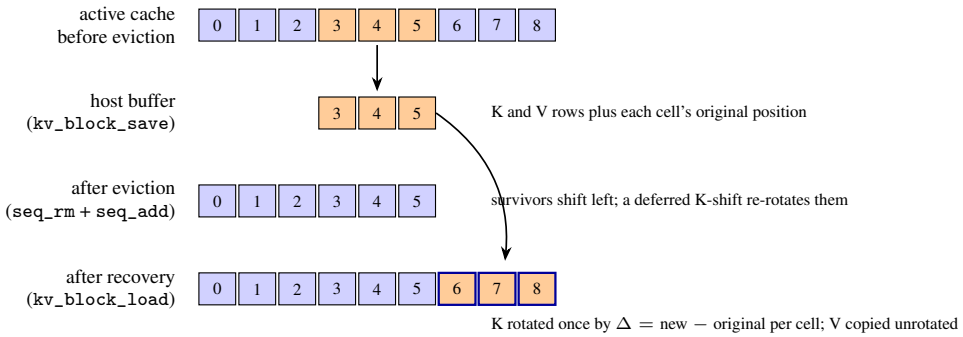

Figure 4: Lifecycle of an evicted block. `kv_block_save` serializes the block's K and V rows with their original positions. Eviction is `seq_rm` plus `seq_add`: survivors shift left to stay contiguous, with llama.cpp's deferred K-shift applying the position change as one RoPE rotation. On recovery, `kv_block_load` writes the saved rows at a new tail position and registers a per-cell rotation by the position delta. No step runs a model forward pass.

4 Design

Figure 3 shows the system. A Python policy layer owns all decisions (what to evict, what to recover, when); a small set of C primitives in a llama.cpp fork owns the mechanics (moving and re-anchoring K/V bytes). This section covers each, bottom up.

4.1 The splice primitives

Two primitives, implemented in `src/llama-context.cpp` and exposed in `include/llama.h` (signatures verbatim in Appendix D), save a range $[p_0, p_1)$ of cells from a sequence into a host buffer and restore it at a new start position. Restoration must produce the same K and V tensors the model would attend to if the restored positions had been decoded in place. Figure 4 traces the lifecycle.

`llama_kv_block_save` reuses the existing `state_write_*` paths filtered to cells whose position is in $[p_0, p_1)$, reading each cell's K and V row plus its original position. `llama_kv_block_load` writes the saved rows into a fresh slot at the new start and re-anchors RoPE by setting $\texttt{shift}[i] = \texttt{new\_pos} - \texttt{original\_pos}$ per cell before running the K-shift graph llama.cpp already uses for its own `seq_add` path. V carries no positional encoding and is copied unrotated.

The correctness of both eviction and recovery rests on one property of RoPE (Su et al., 2024): position enters K as a multiplicative phase, and $K(\text{pos}) \cdot Q(\text{pos}_q)$ collapses to a function of the relative offset, so moving a K row from position p to $p + \delta$ is exactly one rotation per dimension pair. Eviction is therefore a topological cut, not partial erasure: after the deferred shift, surviving dot products are identical to a cache in which the evicted range never existed, and the failure

mode of a wrong eviction is information loss (recoverable), not corruption. The per-block cost is $O(\texttt{n\_cells} \times \texttt{n\_layers} \times \texttt{n\_embd\_kv} \times \texttt{sizeof(dtype)})$ of memcpy plus the rotation.

Flash Attention is a performance precondition. With Flash Attention (Dao et al., 2022; Dao, 2023) disabled, V is stored transposed and the V loop degenerates to 14,336 synchronous CUDA launches per load on Qwen 2.5 7B; a 20-token `kv_block_load` takes 133 ms instead of 0.48 ms. The splice is functionally correct either way, but the usable surface is Ampere-or-later CUDA with FA on, the same surface modern paged-attention stacks target by default. CPU-only and older Vulkan/Metal backends are a real exclusion.

Attention-weight capture. The eviction scorer can use the model's own attention as a relevance signal, but FA fuses the softmax and never materializes weights. Rather than editing the FA kernel across four backends, the fork computes a second softmax in parallel for chosen layers from the same Q and K tensors FA consumes, and copies it to a host buffer after each decode. The main path is unchanged (output tokens are identical with capture on or off), the side product is validated against non-FA mode below f32 rounding tolerance, and the cost is $\sim$1.8 MB per layer slab per step on Qwen 2.5 7B. The C API is in Appendix D.4.

Cross-architecture reach. The fork resolves a context's memory object to its attention KV cache and unwraps hybrid (Mamba+Attention) and iSWA wrappers, so the same save/load/re-anchor path reaches the attention component of hybrid models; the fixed-size recurrent state needs no eviction and is carried forward untouched. For mrope models (Qwen 3.5 and later) the upstream shift guards are removed and the temporal position re-anchors as a whole-vector rotation. For iSWA models the save/load buffer covers both base and SWA sub-caches. Two attention-only primitives (`llama_kv_block_seq_rm`/`_seq_add`) bypass the hybrid wrapper, since the recurrent half rejects partial rollback. Commit-level detail is in Appendix D.

4.2 The policy layer

The policy layer (`evoke/manager.py`, `evoke/scorer.py`, `evoke/recovery.py`) is pure Python; the engine is behind a protocol, so every policy decision is testable without a GPU.

Eviction. Each block carries a score in $[0, 1]$; when the active set crosses $\texttt{high\_watermark} \cdot b$, the lowest-scoring blocks evict down to the low watermark. The score blends up to five signals (the model's own attention mass from the capture primitive, recency, embedding coherence with the current task focus, a recovery-strength term that protects recently recovered blocks from immediate re-eviction, and the forward-looking workspace probe of Section 5.4, computed once as a block's tokens pass prefill), lifted by source-type floors and scaled by a harness-supplied priority. Sink blocks (the first `sink_count` positions, the StreamingLLM anchor) never evict. The factorial in Appendix A.4 attributes most of the policy-side gain to one decision: scoring blocks against the raw user message through a dedicated retrieval embedder (`bge-small-en-v1.5` (Xiao et al., 2024c)) rather than pooled LM hidden states (+72 pp on NIAH at b=512), with a conditional +20 pp from splicing recovered blocks before the new tail decodes. The scorer is the frame the primitive composes in, not the contribution itself.

Recovery. Three backends share one protocol and make the recovery axis ablatable. `discard` drops evicted content outright. `breadcrumb` keeps a compact marker (key, token count, eviction step) so the harness can choose to re-fetch and re-decode. `kv_restore` saves K/V via `kv_block_save` into an LRU host pool (with optional disk spill) and splices on demand. On each user turn a top-K selector (default K=4) scores saved blocks against the incoming message and recovers the best matches before the tail decodes.

Identity-keyed substitution, similarity-shaped selection. EVOKE is a hybrid by construction, and the line matters. Selection (which evicted block to bring back) may use similarity: the top-K selector embeds blocks and queries and picks by cosine, and a harness can instead name keys directly. Substitution (what enters the cache) is identity-keyed: every block carries a stable key such as `file:config.py#0`, and recovery pastes back the exact K/V the model computed at first decode, never a re-encoding or a similarity-retrieved alternative. RAG (Lewis et al., 2020) substitutes

re-encoded text and severs continuity with how the tokens were first attended; EVOKE's systems content lives at the substitution layer.

4.3 Serving

An OpenAI-compatible server (`evoke/server.py`) makes the session, not the request, the unit of state: incoming messages prefix-match against the session's cached tokens and only the new tail decodes; a session pool holds per-session KV snapshots so one model instance serves many concurrent agents. Harnesses can pass per-block hints (`evoke_priority`, `evoke_pinned`, `evoke_task_boundary`) that default to neutral. Server internals, the thinking-trace state machine, and concurrency measurements (16 sessions, 80/80 probes, p999 within 0.13 s of p99) are in Appendices A.10 and B.

5 Evaluation

5.1 Setup

All numbers come from one GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Flash Attention on); every number traces to a script in `scripts/` and a results file in `results/`, with invocations in Appendix C.

Model	Architecture	Coverage
Qwen 2.5 7B Instruct	pure attention, RoPE, Q4_K_M	full suite, budgets 512/1024/2048
Llama 3.1 8B Instruct	pure attention, RoPE, Q4_K_M	full suite, budgets 512/1024/2048
Qwen 3.5 9B	hybrid Mamba/Attention, mrope, thinking	NIAH + multi-fact at b=1024 only (thinking-mode generation cost)
Qwen 3.6 35B-A3B	MoE, mrope, thinking, IQ2_M	NIAH + multi-fact at b=1024 only (IQ2_M sits near the 16 GB ceiling)

Table 2: Models evaluated. The two pure-attention models get full budget sweeps; the hybrid and MoE models are swept at one budget, so cross-architecture claims in this paper are claims at b=1024.

Baselines are reimplemented on the EVOKE substrate per their published configurations, so comparisons isolate policy from engine: recency, StreamingLLM (Xiao et al., 2024a) (sinks + sliding window), H2O (Zhang et al., 2023) (cumulative attention, 10 % recent-tail guard), SnapKV (Li et al., 2024) (observation-window snapshot), InfLLM (Xiao et al., 2024b) (block memory, top-K=8 similarity recall, spliced via the same `kv_block_load`), plus EVOKE's own recovery-less arms (`discard`, `breadcrumb`) as ablations, and an ArkVale-style recall arm (Section 5.5).

5.2 Recovery is the dividing line

Across three recall benchmarks of increasing difficulty, the presence of a recovery path separates passing policies from failing ones, regardless of how smart the eviction selector is. The divide is structural, and we state its construction plainly: each bench plants content that budget pressure forces out of cache before the probe, so it quantifies what the recovery primitive enables, not whether EVOKE's eviction scorer beats H2O's or SnapKV's at choosing victims. The within-cluster comparison where the scorer is on trial is Section 5.3.

Agent-bench: a planted fact past recency. `scripts/agent_bench.py` plants a fact in an early "config file" read, buries it under ten unrelated file reads, then probes it. The bench is deliberately an oracle: it recovers exactly the key `file:config.py` before the probe, isolating primitive cost from selection. Nine strategies at three budgets on Qwen 2.5 7B: every recovery-less policy (recency, StreamingLLM, `discard`, H2O, SnapKV) fails at every budget; every recovery-bearing policy (`breadcrumb`, `kv_restore`, attention-scored EVOKE, InfLLM) passes at every budget. The workspace-scored policy added in this revision is the documented exception, and it sharpens the claim rather than breaking it: `jlens` passes every budget with `discard` recovery by never evicting the fact in the first place (Section 5.4), so the divide is precisely about content the selector has already let go. Llama 3.1 8B reproduces the same 27-cell divide. Recovery cost separates within the passing cluster:

`breadcrumb` re-decodes at 15–17 ms per recovery, `kv_restore` splices at 3–4 ms. Per-cell detail is in Appendix A.1.

Needle-in-a-haystack. `scripts/niah_bench.py` plants one of five needles at one of five depths in a 40-paragraph haystack (∼4× the 1024-token budget); 25 cells per (model, policy, budget).

	Qwen 2.5 7B			Llama 3.1 8B		
policy	b=**512**	b=**1024**	b=**2048**	b=**512**	b=**1024**	b=**2048**
recency	20 %	20 %	40 %	0 %	0 %	0 %
streaming_llm	0 %	20 %	20 %	12 %	20 %	24 %
evoke_discard	0 %	20 %	20 %	12 %	20 %	24 %
evoke_breadcrumb	0 %	20 %	20 %	12 %	20 %	24 %
h2o	20 %	20 %	40 %	20 %	20 %	40 %
snapkv	20 %	20 %	40 %	44 %	68 %	84 %
infllm	**96 %**	**96 %**	80 %	**84 %**	76 %	68 %
evoke_kv_restore	**96 %**	**100 %**	**100 %**	76 %	**88 %**	**88 %**
evoke_attention	**96 %**	**100 %**	**100 %**	76 %	**88 %**	**88 %**

Table 3: NIAH pass rate, 25 (needle, depth) cells per entry. Recovery-less policies pass only when the needle happens to sit in the never-evicted recent tail. SnapKV on Llama is the one documented exception: heavy-hitter retention preferentially keeps a single high-attention needle resident at looser budgets; it falls back to the recovery-less cluster on multi-fact (Table 4), where attention shifts across five topics and no single block holds top-K mass throughout.

Multi-fact, and the divide across architectures. `scripts/multifact_bench.py` plants five facts at jittered depths and probes all five in sequence, churning the cache through five topic shifts; scoring is string-match with an LLM-judge fallback for ambiguous answers. On the n=5 pilot at b=1024 on Qwen 2.5 7B, recovery-less policies score 0–4 % while the recovery-bearing cluster reaches 48–64 %. The divide repeats at b=1024 on the other architectures (Figure 5): Qwen 3.5 9B (hybrid, thinking) 68 % [48.4, 82.8] versus 0–8 % for every recovery-less baseline, and Qwen 3.6 35B-A3B (MoE, IQ2) 52 % [33.5, 70.0] versus 0 %, with the lower absolute number plausibly the aggressive quantization.

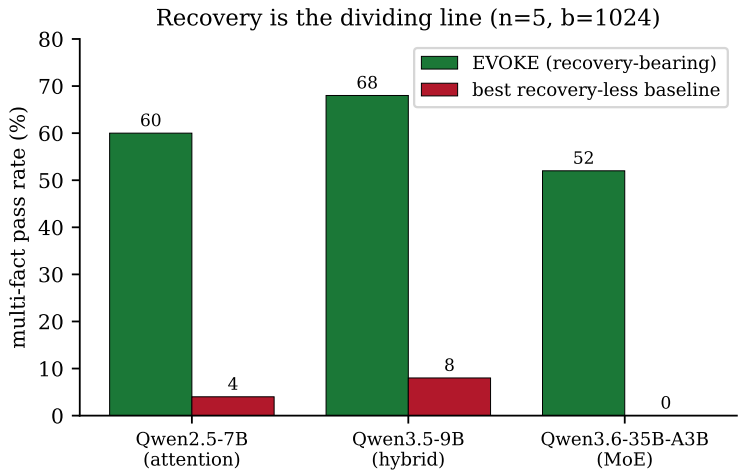

Figure 5: Recovery is the dividing line across architectures (multi-fact, n=5, b=1024). The best recovery-less baseline collapses to 0–8 % on each architecture while the recovery-bearing policy passes; the absolute level varies with architecture and quantization.

5.3 Two recovery-bearing schedulers: a budget crossover, not a winner

InfLLM sits on EVOKE's side of the divide: both treat eviction as reversible, both splice the model's own K/V back through `kv_block_load`; they differ on scheduling. EVOKE evicts by watermark with a multi-signal scorer and recovers top-K=4; InfLLM evicts to a sinks-plus-local-window footprint and recovers a wider top-K=8 by pure retrieval. Extending multi-fact to 15 seeds at three budgets (75 facts per cell) on both fully-swept architectures gives the head-to-head in Table 4 and Figure 6.

strategy	b=**512**	b=**1024**	b=**2048**
Qwen 2.5 7B			
infllm	**81.3 % [71.1, 88.5]**	58.7 % [47.4, 69.1]	56.0 % [44.7, 66.7]
evoke_kv_restore	50.7 % [39.6, 61.7]	57.3 % [46.1, 67.9]	65.3 % [54.1, 75.1]
evoke_attention	49.3 % [38.3, 60.5]	58.7 % [47.4, 69.1]	**65.3 % [54.1, 75.1]**
Llama 3.1 8B			
infllm	**81.3 % [71.1, 88.5]**	65.3 % [54.1, 75.1]	56.0 % [44.7, 66.7]
evoke_kv_restore	58.7 % [47.4, 69.1]	57.3 % [46.1, 67.9]	65.3 % [54.1, 75.1]
evoke_attention	60.0 % [48.7, 70.3]	57.3 % [46.1, 67.9]	**66.7 % [55.4, 76.3]**
recency	0.0 % [0.0, 4.9]	0.0 % [0.0, 4.9]	0.0 % [0.0, 4.9]
streaming_llm	0.0 % [0.0, 4.9]	1.3 % [0.2, 7.2]	13.3 % [7.4, 22.8]
h2o	0.0 % [0.0, 4.9]	8.0 % [3.7, 16.4]	29.3 % [20.2, 40.4]
snapkv	1.3 % [0.2, 7.2]	16.0 % [9.4, 25.9]	42.7 % [32.1, 53.9]

Table 4: Multi-fact pass rate at n=15 (75 facts per cell), Wilson 95 % CIs. Recovery-less baselines shown once for the Llama sweep; the Qwen sweep produces the same cluster. At b=512 InfLLM separates from EVOKE on both architectures (non-overlapping CIs); at b=1024 the policies tie; at b=2048 EVOKE leads with overlapping CIs (directional, not separated).

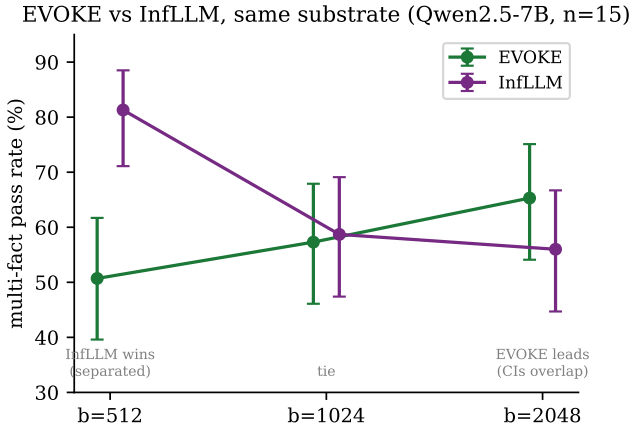

Figure 6: EVOKE versus InfLLM on the same recompute-free substrate (Qwen 2.5 7B, multi-fact, n=15, Wilson 95 % CIs). The schedulers trade places along the budget axis.

InfLLM wins outright at b=512. The per-fact failure cross-tab locates the gap in selection: on the structurally hardest fact, EVOKE's K=4 misses 91 % of cells against InfLLM's 67 %, so the wider per-turn recall net catches more of what tight budgets evict. At b=2048 the ranking reverses directionally: budget headroom lets EVOKE's watermark eviction keep more of the right blocks resident in the first place, where InfLLM's fixed aggressive footprint cannot use the headroom. Both ride the same splice; K and scheduler aggressiveness are budget parameters, and we report the crossover so deployers tune them rather than read either scheduler as uniformly better.

5.4 A forward-looking workspace signal for eviction

Every signal in Section 4.2 either looks backward (attention mass records what past queries already touched; H2O and SnapKV are the eviction-side instances) or is content-agnostic (recency, coherence). None can rank a block before the model has attended to it, which is exactly the regime of eviction pressure during prefill. Gurnee et al. (2026) show that verbalizable representations concentrate in a small per-layer "workspace" subspace read out by a fixed Jacobian lens, that multi-step computation routes through this subspace while routine processing bypasses it, and that a top slice of broadcast heads relays it across positions. If workspace content is what later computation reads from, then *whether a block holds workspace content* is an eviction signal available the moment the block is written. This section validates that signal offline against attention oracles, distills it to a per-position dot product, and runs it as `w_jlens` on the bench.

Offline validation, pre-registered. On 36 planted-fact episodes in the agent-bench distribution (Qwen 2.5 7B-Instruct in 8-bit, 3.7K–15K tokens, fact depths 0.2/0.5/0.8), we compute per-position workspace statistics of the lens readout (excess kurtosis, top-k mass, transport-norm ratio) at three mid-band layers (19/21/23 of 28), aggregate per 128-token block, and score each signal by AUC against the fact block. The kill gate was registered before the runs: a $\geq$2-point AUC lift over recency+SnapKV, or a masked-eviction win at some budget, else the direction stops. The best workspace statistic (layer-23 kurtosis, block mean) reaches fact-AUC **0.891** against SnapKV-window 0.622 and recency 0.481; adding it to the combined score moves AUC from 0.420 to 0.914. Under masked eviction retaining the top 25 % of blocks per signal, the fact stays answerable in **35/36** episodes for the workspace ranking versus 25/36 (SnapKV) and 8/36 (recency), with disjoint Wilson 95 % intervals ([0.858, 0.995] vs. [0.531, 0.820]); at 50 % retention 36/36 vs. 34/36, saturating for both at 75 %. Two boundaries observed: leave-one-block-out logprob drops correlate with no signal (Spearman $|\rho| \leq 0.15$ for all, workspace included), so the signal localizes task-critical content rather than ranking global causal importance; and the advantage concentrates at fact depths 0.2/0.5, since at 0.8 the recency-family baselines keep the fact inside their protected tail anyway. Figures 7 and 8 show both measurements side by side with the Qwen3-8B replication below.

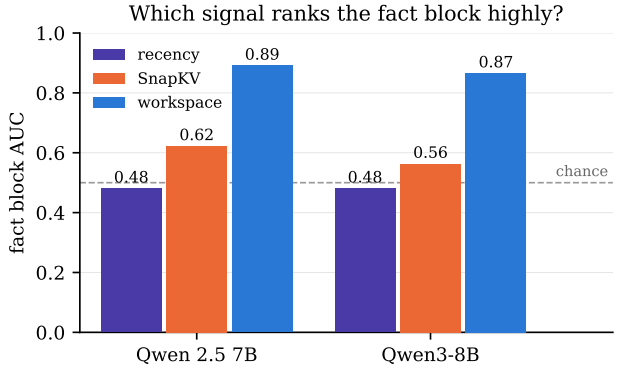

Figure 7: Offline fact-block AUC of eviction signals on both models (36 planted-fact episodes each). Attention history (SnapKV) barely clears chance-level recency; the workspace readout separates cleanly, and the pattern replicates without hand-tuning on Qwen3-8B.

Distillation to a dot product. Lens scoring needs a vocabulary-sized matmul per position, too heavy inside a decode loop, so each (layer, statistic) is distilled into a ridge probe: prediction $= h \cdot w + b$ on the residual entering the scoring layer, fit on 27 episodes and gated on the 9 held out (one per length$\times$depth cell). The winning statistic distills at held-out per-episode mean Spearman **0.959** (block-level 0.953, worst episode 0.952; acceptance bar 0.8, pre-registered), and every one of the nine (layer, statistic) probes exceeds 0.92. The artifact is 137 KB; the fork's layer-input capture (Appendix D) exposes the residuals, so the online cost is one 3584-wide dot product per position per scored layer.

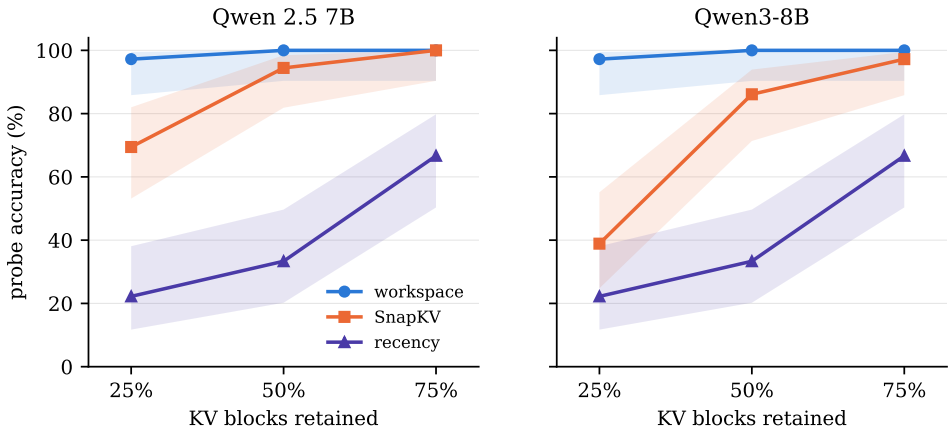

Figure 8: Probe accuracy after masked eviction at three retention budgets (36 episodes, Wilson 95 % bands). At 25 % retention the workspace ranking keeps the fact answerable in 35/36 episodes on both models while SnapKV falls to 25/36 (Qwen 2.5 7B) and 14/36 (Qwen3-8B).

On the bench. `jlens` runs the probe as the sole eviction signal with `discard` recovery and the same hard-eviction and tail-guard pattern as the H2O and SnapKV rows; `jlens_h2o` mixes it 50/50 with cumulative attention. Same protocol as Section 5.2, Qwen 2.5 7B Q4_K_M:

policy (no recovery)	b**=512**	b**=1024**	b**=2048**
recency	fail	fail	fail
h2o	fail	fail	fail
snapkv	fail	fail	pass
jlens	**pass**	**pass**	**pass**
jlens_h2o	**pass**	**pass**	**pass**

Table 5: Planted-fact probe on agent_bench with recovery disabled: the workspace probe keeps the fact resident at budgets where every attention-history selector evicts it. Single probe per cell; the per-cell log is `results/agent_bench_jlens_2026-07-09.txt`.

Decode cost is measured directly rather than inferred from whole-run wall time: generating 128 tokens after a 601-token prefill with capture plus per-step probe scoring over 10 blocks runs at 115.6 tok/s against a 114.9 tok/s baseline (-0.6 %, i.e. noise; `scripts/bench_capture_overhead.py`). The per-step cost is a 28 KB host copy and ten dot products; the one-time prefill cost is probe scoring over the context, $\sim$0.1 s on a 2.3K-token session.

Replication on Qwen3-8B. The full offline pipeline, rerun end to end on Qwen3-8B (36 layers, dense attention, released lens) with no hand-tuning, reproduces the pattern: the automated band procedure picks layers 29–31, the same statistic family wins (layer-30 kurtosis, block mean), fact-AUC reaches 0.865 against SnapKV 0.563 and recency 0.481 (combined lift 43.7 points), masked eviction at 25 % retention keeps the fact answerable in 35/36 episodes versus 14/36 (SnapKV) and 8/36 (recency), and the probe distills at held-out Spearman 0.944. All 36 unmasked probes answer correctly once Qwen3's thinking mode is bypassed for probe generation by forcing an empty think block (the same forced prefix conditions the oracle's answer-logprob measurements, so labels stay comparable across models).

Scope, stated plainly. The probe was fit on the same episode family the bench draws from, so Table 5 shows transfer across quantization (8-bit HF to Q4 GGUF) and engine (HF transformers to llama.cpp), not across task distributions; a LongBench-family evaluation is unrun. The live-bench rows exist for Qwen 2.5 7B only: running them on Qwen3-8B requires thinking-aware generation budgets in the bench harness first. The signal localizes probed facts, not general importance (the LOBO null above), and single-fact retention is the friendliest case for it: under multi-topic churn

where the true working set exceeds the budget, something must still be evicted, and recovery (Section 5.2) remains the mechanism that makes that reversible. The production composition, `jlens` scoring plus `kv_restore` recovery, is wired but not yet swept.

5.5 Where should a recovered block land?

The splice re-anchors recovered blocks to a new contiguous tail position; ArkVale recalls evicted pages at their original positions. To isolate that one axis we force-evict and force-recover 12 facts and vary only placement: `compact` re-anchors to the tail, `sparse` splices back at the original position. Two never-evicted controls bound the result: `no_evict` (facts stay at original positions) and `no_evict_tail` (facts decoded natively at the tail). Qwen 2.5 7B, 128K context, 8 seeds (`scripts/multifact_position_bench.py`).

distance (tokens)	compact (tail)	sparse = no_evict (orig. pos.)	no_evict_tail (native)
16,384	1.00	1.00	1.00
81,920	0.97	0.70	1.00
98,304	0.83	0.74	1.00
114,688	0.77	0.65	1.00

Table 6: Multi-fact recall by recovered-block placement (Qwen 2.5 7B, 12 facts, 8 seeds). `sparse` reproduces `no_evict` exactly, so original-position recovery is lossless. `compact` beats `sparse` only at 80K+ and stays below the native-tail ceiling at every distance.

Three readings. First, `sparse` matches `no_evict` to two decimals at every distance: original-position recovery is faithful, and the comparison arm is not crippled. Second, tail re-anchoring pays only near the context edge: all arms are equal at and below 64K, and `compact` beats `sparse` at 80K+ (0.97 vs 0.70 at 82K). Repeating the sweep on Qwen 2.5 3B inside its 32K window reverses the sign (`compact` 0.86–0.92 versus 1.00 for `sparse` and both controls): a short-context model never reaches the distances where tail placement pays, so re-anchoring is pure cost there. The likely mechanism is positional: lost-in-the-middle bias (Liu et al., 2023b; Hsieh et al., 2024) makes far positions poorly attended, and moving content to the well-attended tail dodges that weakness rather than improving KV fidelity. Third, `compact` stays below `no_evict_tail`'s flat 1.00, and the gap grows with distance: a relocated block's K and V still encode the context it attended at its original position, so relocation of co-attended KV carries a staleness cost that placement cannot remove. Positions are provably clean and misses do not track the shift delta, so this is not a RoPE error; it is the open problem named in Section 8.

An ArkVale-style recall arm. Using two further fork primitives (per-decode query and key capture at a scoring layer), we built an ArkVale-style recall policy on our substrate: per-block bounding-volume key digests, cuboid-mean importance estimation against the live query, and bounded top-K recall-and-evict at original positions. On multi-fact at Qwen 2.5 7B it trails both embedding-recall arms at every budget (Table 7); in a 10-fact diagnostic at b=4096, where the document mostly fits, it reaches 70 %, so the tight-budget gap is recall selection and placement, not a broken pipeline. This ablates two axes (importance-digest versus embedding selection, original-position versus tail placement) on an agent-session workload; it is not a reproduction of ArkVale, which selects pages per layer, a structure a whole-sequence cache cannot represent. We make no claim of advantage over ArkVale's full method.

recall policy	b=512	b=1024	b=2048
ArkVale-style (cuboid-mean, original-position)	0 %	4 %	20 %
`evoke_kv_restore` (embedding, compact)	52 %	60 %	64 %
`infllm` (embedding K=8, compact)	88 %	64 %	60 %

Table 7: ArkVale-style recall ablation on multi-fact, Qwen 2.5 7B (n=5 seeds, 25 facts per cell; the ArkVale arm at b=512 ran 4 seeds, 20 facts). The arm isolates the selection and placement axes on this workload; it is not ArkVale's per-layer method.

5.6 Clearing the context wall at scale

The recall benchmarks fit inside the context window; the motivating failure is the session that does not. We drive a 66,147-token agentic session (150 keyed sections read in sequence, then re-referenced) under an 8,192-token KV budget with a 32,768-token context (`scripts/scale_demo.py`, Qwen 3-14B). Without eviction the cache hits the context wall at section 74 of 150 and the run cannot continue. EVOKE completes the full corpus at $4.1\times$ lower peak KV and recovers re-referenced sections recompute-free; the discard-only arm also stays in budget but pays a $1.11\times$ re-decode tax on re-references (Figure 9). We make no within-window recall claim here.

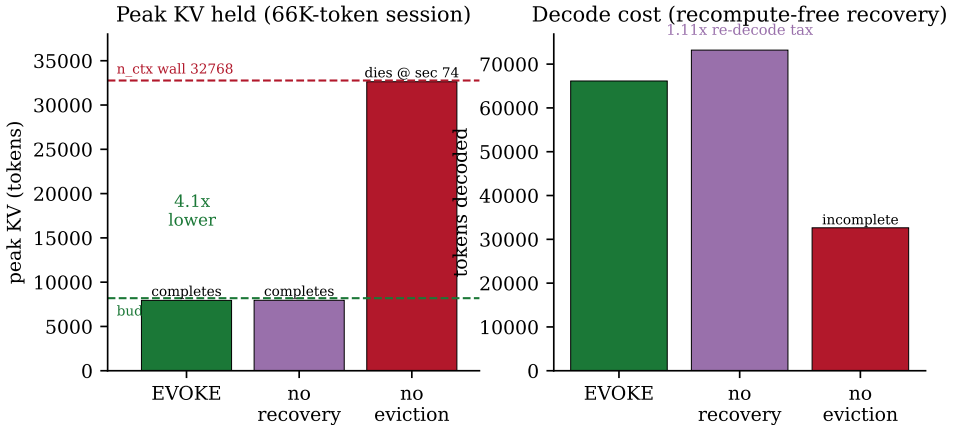

Figure 9: A 66K-token session under an 8K KV budget (Qwen 3-14B). Left: peak KV held; the no-eviction arm dies at the context wall mid-corpus. Right: tokens decoded; EVOKE recovers re-referenced sections without re-decoding them.

5.7 Wall-clock: parity with truncation, not a speed win

A 14-turn planted-fact session through three server policies (n=15 runs, Qwen 2.5 7B, b=1024): `no_eviction` finishes in 18.90 s [18.74, 19.05], `truncate` in 22.11 s [19.46, 24.76], EVOKE in 21.20 s [21.07, 21.33]; EVOKE and truncate are not statistically distinguishable at α=0.05, and all three pass this probe (the fact sits inside truncate's recency window here; agent-bench moves it past recency and truncate fails). EVOKE matches `no_eviction`'s recall at a $\sim$3.6$\times$ smaller cache (684 vs 2476 active tokens) for a $\sim$12 % wall-clock overhead over the stateful-server floor. The claim is recovery at truncation-parity cost, not throughput; the session-length sweep to 56 turns (Appendix A.7) holds the footprint flat and keeps the gap within $\sim$10 %.

5.8 A negative result: the policy does not port to vLLM v1

We ported the primitive and policy to a vLLM v1 fork. The recovery primitive transfers: paged byte transfer plus RoPE re-anchoring composes from existing `swap_blocks_batch` and `rotary_embedding` kernels (numerical deviation 1.6×10^{-2} bf16 MAD). The policy layer does not: in-session recovery needs a logical position space that outlives a single `generate()` call, and the v1 scheduler treats the cache as a contiguous prefix of the active request. What transfers is same-content gap-fill on re-sent history, where a 5/5-both-arms probe cannot distinguish EVOKE from LRU (n=5, no CIs). The finding names a concrete precondition for ports: a session-scoped position space, which llama.cpp has natively and vLLM v1 lacks.

6 Related Work

Eviction-only compression. StreamingLLM (Xiao et al., 2024a) keeps attention sinks plus a recent window; H2O (Zhang et al., 2023) and Scissorhands (Liu et al., 2023) drop low-attention entries; SnapKV (Li et al., 2024) and Ada-KV (Feng et al., 2024) refine the selection. All treat eviction as one-way, and all rank by attention history, so none can score a block before it has been attended to.

Method	Recalls evicted KV	Recall keyed by	Recall placement	Substrate
Recency / StreamingLLM	no	—	—	attention
H2O / SnapKV / Scissorhands	no	—	—	attention
Compressive transformers	no (compress)	—	—	attention
RAG	re-encodes text	similarity	re-encoded	external
InfLLM	yes (no recompute)	similarity	unified splice	attention
ArkVale	yes (no recompute)	importance digest	original position	attention, per-layer
EVOKE	yes (no recompute)	identity / similarity	re-anchor / sparse	attention, hybrid, MoE

Table 8: Where EVOKE sits among KV-cache eviction and recovery methods.

EVOKE adopts the attention signal and the sink anchor, and adds the recovery path that makes a wrong eviction recoverable; H2O and SnapKV run as same-substrate baselines in Section 5.

Interpretability signals for cache policy. Gurnee et al. (2026) identify a small verbalizable workspace subspace, read by a fixed per-layer Jacobian lens, that multi-step computation routes through and broadcast heads relay across positions; the finding is causal (ablating the subspace collapses multi-hop reasoning while leaving single-hop benchmarks intact) but is not framed as a systems signal. Section 5.4 is, to our knowledge, the first use of that structure for KV-cache management: workspace-ness of a block, distilled to a linear probe, as a forward-looking eviction score. It is orthogonal to the recovery axis this paper contributes and composes with it.

Recovery-bearing methods. ArkVale (Chen et al., 2024) is the closest prior work and the origin of recompute-free recall of evicted KV; the regime, addressing, placement, and substrate differences are stated in Section 1, and Section 5.5 measures the placement axis. InfLLM (Xiao et al., 2024b) keeps a block-level external KV memory recalled by similarity; EVOKE benchmarks it as a same-substrate row and shares its block-memory framing while splicing into the unified contiguous cache. CacheFocus (Lee et al., 2025) also re-positions cached keys by recomputing RoPE, but over independently prefilled RAG chunks; EVOKE relocates blocks that were co-attended inside one live sequence, which is exactly where the staleness cost of Section 5.5 appears. Compressive transformers (Rae et al., 2019) summarize old KV; EVOKE trades host RAM for exact recovery instead.

KV save-restore systems. CachedAttention (Gao et al., 2024) reloads a whole inactive session's cache at turn boundaries; CacheGen (Liu et al., 2024b) compresses KV into lossy bitstreams for network streaming; CacheBlend (Yao et al., 2025) reuses per-chunk KV across RAG requests and selectively recomputes a token subset to restore cross-attention. EVOKE differs in granularity and timing (one block at a time, inside a live decode loop, mid-session) and in being byte-exact rather than blended or compressed. MemGPT (Packer et al., 2023) pages text in and out of the prompt; EVOKE pages the tensors themselves.

Cache infrastructure. vLLM/PagedAttention (Kwon et al., 2023), SGLang's RadixAttention (Zheng et al., 2024), LMCache (LMCache Team, 2024), and DistAttention (Lin et al., 2024) govern how cached bytes move and are shared across requests; EVOKE's policy decides which blocks of one session evict and return, an orthogonal concern, and Section 5.8 reports what the port onto a paged substrate needs. RoPE re-anchoring builds on RoFormer (Su et al., 2024); retrieval embeddings use BGE (Xiao et al., 2024c).

7 Discussion and Limitations

Contribution boundary. The dividing-line results quantify what the recovery primitive enables; they do not show EVOKE's heuristic eviction scorer beating published selectors, and Section 5.3 shows a wider-K pure-retrieval scheduler winning outright at tight budgets on the same primitive. The factorial (Appendix A.4) is similarly blunt: one policy decision (a dedicated retrieval embedder over raw message text) carries most of the policy-side gain. This revision adds one selector-side claim with pre-registered gates behind it: the workspace probe of Section 5.4 does outrank H2O's and SnapKV's victim choices on the planted-fact family, on one model so far, with the distribution

caveats stated there. The remaining contribution is unchanged: the reversible-eviction substrate, the identity-keyed addressing, and the measurements.

Relation to ArkVale, stated once. ArkVale owns recompute-free recall of evicted KV into a dense cache, with per-layer paged selection this substrate cannot represent; our ArkVale-style arm ablates selection and placement on an agent workload and is not a head-to-head with the full method. What EVOKE adds is the regime (multi-turn agent sessions with harness-visible block identity), the cross-architecture primitives, and the placement measurement, including the finding that EVOKE's own tail re-anchoring is regime-gated and lossy against a native-tail control.

Staleness of relocated co-attended KV. Every prior recompute-free reuse method sidesteps staleness by construction: ArkVale never relocates, CacheFocus and CacheBlend pre-isolate chunks so there is no cross-context to go stale. EVOKE relocates blocks that attended their original neighborhood, and Table 6 prices that choice: 0.77–0.97 against a 1.00 native-tail ceiling, worsening with shift distance. Whether a repair exists that is cheaper than CacheBlend-style partial recompute is open (Section 8).

Memory accounting. Saved blocks cost real host RAM: $\sim$7 MiB per 128-token block on Qwen 2.5 7B, so a 1000-block session holds $\sim$7 GiB and N co-located sessions pay $N\times$. The LRU bound and disk-spill tier exist for this (degradation under pressure is measured in Appendix A.5); the trade against summarizing schemes is explicit: RAM proportional to evicted content, in exchange for exact recovery.

Hybrid models and thinking traces. On hybrid models EVOKE manages only the attention cache; the fixed-size recurrent state is untouched, and pure-SSM models are out of scope. Stripping a thinking trace from the cache would require partial rollback of the recurrent half, which hybrid memory rejects, so the shipping configuration keeps traces in cache and in returned content; a no-shift eviction mode over the attention-only primitives is the designed fix, not yet wired end-to-end.

Scope of the evidence. The benches are synthetic, constructed so that the probed content is evicted; a real-agent-trace evaluation (SWE-bench Verified, τ-bench, recorded harness sessions) is future work and we do not extrapolate to it. Latency is measured end-to-end on two architectures; the hybrid and MoE models are swept at one budget; the KV-quantization comparison covers llama.cpp's Q4_0 only (Appendix A.8), not KIVI (Liu et al., 2024). Policy quality is bounded by the retrieval encoder; larger encoders are an unrun sweep.

8 Open Problems

Four stand out. **Staleness repair:** a recompute-free or near-free correction for relocated co-attended KV that closes the Table 6 gap to the native-tail ceiling would make placement a free choice; CacheBlend's partial recompute is the cost bar to beat. **A session abstraction for paged substrates:** Section 5.8 reduces the vLLM port to one missing ingredient, a logical position space scoped to a session rather than a request. **Real-agent traces:** the server is wire-compatible with the harnesses that would produce them; the work is the evaluation harness, not a port. **The quantization frontier:** a KIVI-class head-to-head would settle whether aggressive KV quantization substitutes for eviction on this deployment class. Engineering directions (iSWA end-to-end on Gemma, multi-layer attention scoring, learned harness priorities, tensor-parallel validation) are tracked in the repository.

9 Conclusion

EVOKE makes KV-cache eviction reversible for agent sessions: evicted blocks keep their tensors in host RAM, and an identity-keyed, recompute-free splice returns them to the live cache when the agent refers back. On one consumer GPU, the splice is 5.9–7.5$\times$ cheaper than re-prefill over the full lifecycle, restores blocks that work as well as a fresh re-decode, and separates every recovery-bearing policy from every recovery-less one across three recall benchmarks and four model families. The same measurements also bound the idea: a wider recall net beats EVOKE's scheduler at tight budgets on the same primitive, tail re-anchoring pays only near the context edge, and relocated co-attended

KV carries a staleness cost no placement choice removes. Recompute-free recall follows ArkVale; making it identity-keyed, cross-architecture, and measured in the agent regime is what this paper adds.

Code and Reproducibility

The EVOKE policy layer (Python) is at `github.com/Anyesh/EVOKE` under Apache 2.0. The forked llama.cpp with the C primitives is at `github.com/Anyesh/llama.cpp` under MIT (the upstream license). The partial vLLM port is at `github.com/Anyesh/vllm` under Apache 2.0 (the upstream vLLM license). Every number in Section 5 was produced by a script in `scripts/` checked into the EVOKE repository, with exact invocations in Appendix C; raw output files are under `results/`.

References

Gurnee, W., Sofroniew, N., Lindsey, J., et al. (2026). Verbalizable Representations Form a Global Workspace in Language Models. *Transformer Circuits Thread*. `https://transformer-circuits.pub/2026/workspace`.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP 2023*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., & Shrivastava, A. (2023). Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *NeurIPS 2023*.

Rae, J. W., Potapenko, A., Jayakumar, S. M., & Lillicrap, T. P. (2019). Compressive Transformers for Long-Range Sequence Modelling. *arXiv preprint arXiv:1911.05507*.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing 568:127063, 2024*.

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024). Efficient Streaming Language Models with Attention Sinks. *ICLR 2024*.

Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Han, S., & Sun, M. (2024). InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *arXiv preprint arXiv:2402.04617*.

Chen, R., Wang, Z., Cao, B., Wu, T., Zheng, S., Li, X., Wei, X., Yan, S., Li, M., & Liang, Y. (2024). ArkVale: Efficient Generative LLM Inference with Recallable Key-Value Eviction. *NeurIPS 2024*.

Lee, K.-H., Park, E., Han, D., & Na, S.-H. (2025). CacheFocus: Dynamic Cache Re-Positioning for Efficient Retrieval-Augmented Generation. *arXiv preprint arXiv:2502.11101*.

Hsieh, C.-Y., Chuang, Y.-S., Li, C.-L., Wang, Z., Le, L. T., Kumar, A., Glass, J., Ratner, A., Lee, C.-Y., Krishna, R., & Pfister, T. (2024). Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization. *Findings of ACL 2024*.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., & Chen, B. (2023). H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS 2023*.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., & Sheng, Y. (2024). SGLang: Efficient Execution of Structured Language Model Programs. *NeurIPS 2024*.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., & Chen, D. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS 2024*.

Feng, Y., Lv, J., Cao, Y., Xie, X., & Zhou, S. K. (2024). Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. *arXiv preprint arXiv:2407.11550*.

Cheng, J., Liu, Y., Wang, K., Xiang, X., Yu, X., Liu, J., Yi, F., & Jiang, J. (2024). LMCache: 7x Throughput Improvement Through Layered, Cross-Engine KV Cache. *vLLM Production Stack project*.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., & Zuo, P. (2024). Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. *USENIX ATC 2024*.

Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., & Jiang, J. (2024). CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *SIGCOMM 2024*.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., & Jiang, J. (2025). CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. *EuroSys 2025*.

Lin, B., Zhang, C., Peng, T., Zhao, H., Xiao, W., Sun, M., Liu, A., Zhang, Z., Li, L., Qiu, X., Li, S., Ji, Z., Xie, T., Li, Y., & Lin, W. (2024). Infinite-LLM: Efficient LLM Service for Long Context with DistAttention and Distributed KVCache. *arXiv preprint arXiv:2401.02669*.

Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., & Nie, J.-Y. (2024). C-Pack: Packed Resources For General Chinese Embeddings. *SIGIR 2024*.

Gu, A., & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv preprint arXiv:2312.00752*.

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? *COLM 2024*.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., & Li, J. (2023). LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL 2024*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL 2024*.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS 2022*.

Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). KIVI: A Tuning-Free Asymmetric 2-bit Quantization for KV Cache. *ICML 2024*.

Cai, Z., Zhang, Y., Gao, B., Liu, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Chang, B., Hu, J., & Xiao, W. (2024). PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069*.

Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., & Han, S. (2024). Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *ICML 2024*.

Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., & Han, S. (2024). DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. *arXiv preprint arXiv:2410.10819*.

Prabhu, R., Nayak, A., Mohan, J., Ramjee, R., & Panwar, A. (2024). vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437*.

Qwen Team. (2024). Qwen 2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.

A Detailed Evaluations

This appendix collects evaluations that are referenced from the main body but whose full detail would crowd the headline results.

A.1 Per-cell agent_bench (scorer ablation context)

The full per-cell agent_bench table (referenced in Sections 5.2 and 5.7), with eviction counts and per-recovery latency, on Qwen 2.5 7B at `block_size=64`:

budget	strategy	probe	evict	recov	rec_ms
512	recency	fail	35	0	0.00
512	streaming_llm	fail	35	0	0.00
512	evoke_discard	fail	35	0	0.00
512	h2o	fail	34	0	0.00
512	snapkv	fail	34	0	0.00
512	evoke_breadcrumb	PASS	35	0	17.25
512	evoke_kv_restore	PASS	35	1	**3.13**
512	evoke_attention	PASS	31	0	**0.01**
512	infllm	PASS	39	1	3.12
1024	recency	fail	29	0	0.00
1024	streaming_llm	fail	28	0	0.00
1024	evoke_discard	fail	28	0	0.00
1024	h2o	fail	24	0	0.00
1024	snapkv	fail	24	0	0.00
1024	evoke_breadcrumb	PASS	28	0	15.40
1024	evoke_kv_restore	PASS	28	1	**3.86**
1024	evoke_attention	PASS	26	1	3.75
1024	infllm	PASS	35	1	3.76
2048	recency	fail	9	0	0.00
2048	streaming_llm	fail	10	0	0.00
2048	evoke_discard	fail	10	0	0.00
2048	h2o	fail	4	0	0.00
2048	snapkv	fail	4	0	0.00
2048	evoke_breadcrumb	PASS	10	0	15.70
2048	evoke_kv_restore	PASS	10	1	**3.89**
2048	evoke_attention	PASS	8	0	**0.00**
2048	infllm	PASS	31	1	3.07

The `evoke_attention` row at $b{=}512$ is the differential against the heuristic scorer at the tightest budget: with attention scoring, the planted fact block is never evicted in the first place (the model's attention across the unrelated filler turns keeps landing on the config-file block, so the scorer assigns it a high score and eviction picks lower-attended document blocks instead). Total evictions drop by one (35 to 34) and recoveries go to zero, and the probe still passes. At 1024 and 2048 the two strategies converge: recency is sufficient when the budget can hold most history naturally.

A.2 Server eviction demo

A 14-turn session driven directly against the chat-completions API. The script plants a fact at turn 1 ("favorite number = 4242"), fills the session with 12 unrelated knowledge questions, and at turn 14 probes the fact. The recorded demos are kept as visual proof of the per-turn evolution of `active_tokens` against the budget.

Qwen 2.5 7B (pure attention), budget=1024. With smart top-K recovery and the `KVRestoreBackend` RAM budget configured tight (so the LRU spill tier exercises in the same run), the session survives **40 evictions and 13 recoveries** while the model still recalls "4242" at the probe.

Qwen 3.5 9B (hybrid Mamba+Attention with mrope and thinking), budget=1024. The model emits a `<think>...</think>` trace each turn, so per-turn token counts are higher. Hybrid memory rejects partial tail rollback of the recurrent half, which means strict thinking-strip evictions would force a session reset on every turn. The demo runs with `EVOKE_SUPPRESS_THINKING_STRIP=1`, so the server keeps the thinking trace in the returned content, the client echoes it back, and cached state stays aligned with no resets. The session settles at **26 evictions and 4 recoveries**, fact recalled.

A.3 Default-knob ablations

We sweep one policy knob at a time across the full 25-cell NIAH grid (5 needles $\times$ 5 depths) at b=1024 on Qwen 2.5 7B, holding everything else at the recommended `evoke_attention` configuration:

knob	swept values	pass rate per value
`block_size`	{32, 64, 128, 256, 512}	84, **100**, 80, 56, 20 %
`attention_capture_layer`	{4, 10, 14, 20, 24, 27}	all 100 %
`smart_recover_k`	{1, 2, 4, 8, 16}	all 100 %
`w_coherence`	{0.0, 0.2, 0.4, 0.6, 0.8}	all 100 %
`recent_tail_protect_frac`	{0.0, 0.05, 0.1, 0.2, 0.3}	all 100 %

The `block_size` curve is the only knob that moves the pass rate: `block_size=64` hits the headline 100 % pass rate, and degradation is symmetric on both sides. Finer-grained blocks (32) fragment fact-bearing content across two neighbouring blocks; coarser blocks (`128, 256, 512`) dilute the block embedding's representative signal with adjacent haystack content. The other four knobs are flat at 100 % on this workload; tighter-budget or multi-fact workloads (Section 5.3) where eviction pressure is sustained throughout the session would likely surface knob-level differentiation.

A.4 Smart-recovery policy factorial

The smart-recovery loop in Section 4.3 has three named policy decisions: (K1) score using a dedicated retrieval-tuned embedder applied to the raw user-message text, versus pooling LM hidden states; (K2) run the recovery splice before the new user-message tail is decoded, versus after; (K3) gate the top-k recovery against the strongest non-current-turn resident block's similarity, versus skipping. We run the full 2^3 factorial on NIAH at b=512 on Qwen 2.5 7B (25 cells per row).

cell	K1	K2	K3	pass	Wilson 95 % CI
000	off	off	off	28.0 %	[14.3, 47.6]
001	off	off	on	28.0 %	[14.3, 47.6]
010	off	on	off	0.0 %	[0.0, 13.3]
011	off	on	on	0.0 %	[0.0, 13.3]
100	on	off	off	76.0 %	[56.6, 88.5]
101	on	off	on	76.0 %	[56.6, 88.5]
110	on	on	off	96.0 %	[80.5, 99.3]
111	on	on	on	96.0 %	[80.5, 99.3]

Marginals over n=100: K1 (raw-text retrieval embedding) +72 pp (14 % off vs 86 % on); K2 (recover-before-decode) -4 pp main effect but +20 pp conditional on K1 on, -28 pp conditional on K1 off; K3 (resident-gate) 0 pp on this benchmark. K1 is the dominant policy decision by a wide margin: the bge-small retrieval embedder widens the cosine band from 0.85 to 0.93 (LM hidden states) to 0.4 to 0.9, and the top-k selection becomes discriminative enough to pick the needle block over haystack noise. K2's asymmetry has a mechanical explanation: recovering before decode splices the recovered block earlier in the cache than the probe, so the model attends back to it; this is a win when retrieval is good (K1 on) and a regression when retrieval is bad (K1 off, the wrong block lands as earlier context). K3 contributes no measurable effect on this benchmark at b=512. The resident-gate was added to address a depth-95 % failure mode where a top-k recovery brought back a weaker match than the already-resident needle, but this factorial's metric does not expose that mechanism. At the budget and benchmark we ran the factorial against, K3 has no measurable effect.

A.5 Host-RAM pressure: graceful degradation

The four-tier saved-block pool (RAM-resident → disk-spilled → breadcrumb-only → discarded) is exercised under tight host memory by `scripts/hostram_bench.py`: a 20-fact session against an 80-paragraph haystack at b=1024 with `kv_restore_ram_budget_bytes` set to hold only 8 saved blocks (∼29 MiB of K/V on Qwen 2.5 7B), with the disk-spill tier enabled.

Outcome: across the session, 207 blocks are evicted from active KV. 200 of them spill to disk, 4 stay RAM-resident at the end (the LRU pool), and 3 fall through to breadcrumb-only. No block is discarded outright; `lru_drops` = 0 because the spill tier catches every saved-pool eviction. Smart-recovery successfully restores 4 of 20 facts under this ∼14 % RAM ceiling, well below the ∼60 % rate the same workload would hit with unbounded RAM, but the system serves the entire session without crashing and the recovery pipeline reaches the correct stored bytes when it does pick them. The spill tier acts as the slow-path safety net rather than the dominant tier in production deployments configured with a larger RAM budget.

A.6 Persistent-reference (keepalive) ablation

The Appendix A.1 differential is observable only at the tightest budget because the filler turns in agent_bench are semantically unrelated to the planted fact: the model's attention drifts off the config-file block within a few turns and the two scoring policies converge.

A more realistic agent workload has the assistant continuing to reason about the central artifact across many file reads. We repeat the agentic eval with each of the ten filler-file contents referencing "the retry policy from config.py", and the final probe phrased "looking at the retry policy in config.py, what is the maximum retry limit?", a question whose answering tokens have strong attention to the early config block.

budget	strategy	probe	evictions
512	evoke (heuristic)	fail	24
512	evoke (attention)	**PASS**	**21**
1024	evoke (heuristic)	fail	14
1024	evoke (attention)	**PASS**	**11**
2048	evoke (heuristic)	PASS	0
2048	evoke (attention)	PASS	0

This bench is a scorer-only ablation: the harness does not invoke recovery. A block the scorer chooses to evict is gone for the rest of the run. Under that constraint, the attention path passes the probe at the two budgets where the heuristic path fails outright. Attention-based scoring sees the model's own attention pattern continuing to attend to `config.py` through every filler turn and refuses to evict it. The heuristic-only scorer evicts the config block under pressure and the fact is gone before the probe runs. Recompute-free recovery is a second line of defense, but it works best when the eviction policy did not drop the wrong block in the first place.

A.7 Session-length scaling and the recovery-aware fix

The Section 5.7 head-to-head is fixed at 14 turns. The scaling sweep runs the same workload at $T \in \{14, 28, 56\}$ turns, 5 seeds per cell, and produces a paired curve: a baseline curve with the production scorer, and a recovery-aware curve with `w_recovery=1.0`.

EVOKE (either curve) holds the active-token footprint at ∼670 to 720 tokens across the 4× session-length sweep, while `no_eviction` grows linearly to 12 607 tokens at T=56. Wall-clock is comparable to truncate within roughly 7 to 10 % across the sweep, not faster. We do not claim a speed win at long sessions; recovery has a cost and truncate has nothing to recover. EVOKE matches truncate's memory footprint and adds a recovery capability truncate does not have.

The recovery-aware fix (`w_recovery` = 1.0) shuts down a recover-then-immediately-evict thrash: recoveries drop from 24/80/192 to 4/20/20 across T=14/28/56 (-83 to -90 %), and evictions drop in lockstep. Wall-clock does not improve because the retrieval embedder (∼10 ms per query) substitutes for the saved thrash work at these session lengths. The fix is transparent on accuracy: multifact

turns	policy	active	evict	recov	elapsed (s, 95 % CI)
14	no_eviction	2476	0	0	19.19 [18.76, 19.63]
14	truncate	685	66	0	21.33 [20.76, 21.90]
14	evoke (baseline)	684	90	24	21.58 [21.17, 21.99]
14	evoke (recovery-aware)	683	65	4	21.79 [21.54, 22.04]
28	no_eviction	6221	0	0	51.05 [50.10, 52.00]
28	truncate	717	486	0	65.14 [64.35, 65.94]
28	evoke (baseline)	721	566	80	68.06 [67.23, 68.89]
28	evoke (recovery-aware)	616	507	20	67.84 [66.65, 69.03]
56	no_eviction	12607	0	0	106.11 [105.19, 107.03]
56	truncate	674	2316	0	170.41 [169.27, 171.54]
56	evoke (baseline)	664	2522	192	182.36 [180.67, 184.06]
56	evoke (recovery-aware)	675	2400	20	185.73 [184.98, 186.48]

verification shows recovery-aware 48/56/64 % vs kv_restore 52/60/64 % at budgets 512/1024/2048, CIs heavily overlap. The wall-clock parity is structural.

A.8 KV cache quantization on the measured substrate

A reviewer asked whether KV-cache quantization is the cheaper alternative to eviction. We measured llama.cpp's stock `GGML_TYPE_Q4_0` (per-tensor symmetric 4-bit), which is the production-available default in the runtime EVOKE targets. We did not run KIVI (Liu et al., 2024) or other per-channel-per-token asymmetric schemes; the result below speaks only to the Q4_0 substrate.

The Q4_0 result is uniform: NIAH, multifact, and agent_bench all collapse to 0 % at every budget. Representative agent_bench generation at b=2048 with full 5962-token context preserved at Q4 precision: ‘‘`, and, and, and, and, and, and, and, and...`’’. F16 EVOKE at b=1024 retains 96 to 100 % NIAH on the same workload. On this substrate (Q4_0, Qwen 2.5 7B) 4-bit KV quantization is not a usable alternative to eviction. We do not extrapolate this to KIVI or other per-channel asymmetric schemes, whose per-channel structure is specifically designed to keep generation viable at lower bit-widths; running them is the next experiment for a deployer choosing between policy-layer eviction and aggressive KV quant.

A.9 Llama 3.1 8B: cross-architecture detail

NIAH and multifact pass-rates on Llama 3.1 8B are included in the main tables (Tables 3 and 4). A capture-layer ablation on the same NIAH grid (`EVOKE_ABL_DIM=attention_layer`, values $\{8, 16, 20, 24, 28\}$) yields identical pass-rates at both budgets: 88 % [70.04, 95.83] (22/25 cells) across all five capture depths at b=1024, and 76 % [56.57, 88.50] (19/25 cells) across all five capture depths at b=512, with the same cells failing at every layer in each budget. Neither the 12-pp Qwen-vs-Llama gap at b=1024 nor the 20-pp gap at b=512 is a capture-layer tuning miss; both reflect architectural differences in Llama's attention patterns or selection-step bottlenecks. The retrieval encoder picks the same residency set regardless of which capture layer feeds the eviction scorer. This validates the "ports without modification" claim: capture-layer choice has no measurable effect on Llama at any budget we tested.

The three EVOKE variants (`kv_restore`, `attention`, `recovery_aware`) converge to identical NIAH pass rates (76/88/88 %) confirming the recovery-aware fix is transparent on single-needle retrieval. Agent_bench on Llama 3.1 8B: EVOKE strategies (breadcrumb, kv_restore, attention) and InfLLM all PASS the planted-config probe at every budget; recency, streaming_llm, evoke_discard, h2o all fail (the same divide as Qwen).

A.10 Concurrency and PCIe bandwidth

We drive $N \in \{1, 4, 8, 16\}$ concurrent sessions through the same evoke_serve instance (Qwen 2.5 7B, b=1024, smart-recovery on, 14-turn planted-fact workload per session). Each level repeats for five rounds so the per-turn latency distribution at N=16 is built from 1120 turn samples.

Two findings the single-shot runs did not surface. The per-turn latency distribution is tightly banded under sustained load: at every N, the p999 latency is within 0.13 s of the p99 latency. There is no

N	probe_ok	n_turn	wall (s)	p50	p95	p99	p999	max
1	5/5	70	38.9	2.71	3.04	3.10	3.10	3.19
4	20/20	280	51.5	3.56	4.52	4.87	4.87	4.90
8	40/40	560	98.8	6.89	8.95	9.10	9.14	9.16
16	80/80	1120	205.0	14.86	17.77	18.01	18.13	18.14

Table 9: Per-turn latency in seconds at varying concurrency. Probe pass is 100 % at every N; per-round wall-clock at $N{=}16$ varies by under 0.3 % across the five rounds.

long tail; the latency band is set by the GPU's per-decode compute time. The 100 % probe-OK rate holds across all 80 $N{=}16$ sessions, so the K/V splice primitive holds up to 16 concurrent evicting sessions without correctness loss.

Per-session wall-clock scales roughly linearly with N ($\sim$23$\times$ at 16$\times$ concurrency). The dominant scaling factor is GPU compute contention (a single 16 GB GPU runs one Qwen 2.5 7B forward pass at a time), not the PCIe save/load path. Back-of-envelope: at 56 KiB per token times 128-token blocks ($\sim$7 MiB per block), N simultaneous eviction events of k blocks each move $7Nk$ MiB across PCIe Gen4 16 GB/s, $\sim$0.4 ms per block per session; at $N{=}16$ and a 4-block eviction burst the aggregate PCIe traffic is 448 MiB at $\sim$28 ms serial worst case. The PCIe pessimism is not the binding constraint here; per-GPU compute throughput is. Production multi-tenant scaling would need tensor-parallel or paged-virtual KV layout to lift the compute ceiling, which this paper does not claim to provide.

B Server Detail

The chat-completions API is stateless: every request resends the full history, and the canonical response is to re-prefill from scratch or, with prefix caching, reuse the unchanged prefix. EVOKE's server (`evoke/server.py`, `evoke/session.py`) makes the session the unit instead. A persistent `Session` with one `EvokeManager` outlives every request: incoming messages are templated and tokenized, prefix-matched against the session's `cached_tokens` to find the longest prefix already resident, and the tail past that point is decoded through `add_context_tokens`, which creates blocks and fires watermark eviction. The SSE stream runs a three-state machine (thinking, tool-locked, normal) that strips `<think>` traces and parses `<tool_call>` XML into OpenAI's `tool_calls` shape.

A `SessionPool` lets one model in VRAM host many concurrent sessions, each with its own KV identity: it holds per-session Python state plus a serialized engine snapshot, and on an `X-EVOKE-Session` switch serializes the outgoing KV state and restores the incoming snapshot in one memcpy proportional to active KV size with no recompute, evicting the least-recently-touched inactive session past `max_sessions` (default eight). `scripts/verify_multi_session.py` confirms two interleaved sessions retrieve their own planted facts without cross-contamination. The session pool swaps KV state but keeps model weights as one VRAM instance, so concurrency is bounded by per-session KV footprint: a 7B Q4_K_M model takes $\sim$5 GB, leaving $\sim$11 GB on a 16 GB GPU for active KV, swap-in state, and FA working memory.

Smart recovery. The session runs smart top-k recovery on each user turn (default $k{=}4$, cost capped at k `kv_block_load` calls), scoring saved blocks against the raw user-message text through the retrieval embedder and splicing the chosen blocks back before the new tail decodes. The factorial in Appendix A.4 attributes the policy-side gains to these two choices.

C Reproducing the Numbers

All numbers in Section 5 were produced on a single GPU host (RTX 4070 Ti SUPER, 16 GB VRAM, CUDA 13.1, Python 3.12) against Qwen 2.5 7B Instruct (`Qwen2.5-7B-Instruct-Q4_K_M.gguf` from the official Qwen GGUF release). The EVOKE policy layer runs in Python; the C primitives live in the forked llama.cpp.

C.1 Build the fork

```
# Clone the EVOKE-modified llama.cpp fork
git clone https://github.com/Anyesh/llama.cpp.git llama.cpp
cd llama.cpp

# Build with CUDA + shared library output
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
# Output: build/bin/libllama.so
```

C.2 Install the EVOKE Python package

```
git clone https://github.com/Anyesh/EVOKE.git evoke
cd evoke
uv sync --extra server

# Point the engine binding at the EVOKE fork build
export LLAMA_CPP_LIB=/abs/path/to/llama.cpp/build/bin/libllama.so
export EVOKE_MODEL_PATH=/abs/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf
```

C.3 Section 3.3 latency table (`profile_recover.py`)

```
uv run python scripts/profile_recover.py
```

The script does five warmup cycles at 40 tokens to settle CUDA graph compilation, then sweeps block sizes 20, 40, 80, 160, 320, 640, 1280 with five trials each, reports the mean save time, mean load time, and mean re-prefill time. The numbers in Section 3.3 are the means.

C.4 Appendix A.2 eviction demo (`eviction_demo.py`)

```
# In one terminal: start the server
LLAMA_CPP_LIB=$LLAMA_CPP_LIB EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_HOST=0.0.0.0 EVOKE_BUDGET=1024 EVOKE_MODEL_NAME=qwen25 \
uv run python scripts/evoke_serve.py

# In another terminal: drive the demo
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/eviction_demo.py
```

For the Qwen 3.5 hybrid run, add `EVOKE_SUPPRESS_THINKING_STRIP=1` to the server env and load `Qwen3.5-9B-Q4_K_M.gguf`.

C.5 Section 5.7 head-to-head baselines (`baseline_bench.py`)

```
EVOKE_SERVER='http://eval-host:8000' \
EVOKE_SSH_HOST='user@eval-host' \
EVOKE_LIB_PATH='/path/to/libllama.so/on/eval/host' \
EVOKE_MODEL_PATH='/path/to/Qwen2.5-7B-Instruct-Q4_K_M.gguf' \
EVOKE_BUDGET=1024 EVOKE_N_CTX=16384 EVOKE_MODEL_NAME=qwen25 \
  uv run python scripts/baseline_bench.py
```

The bench drives three policies (`no_eviction`, `truncate`, `evoke`) over the same 14-turn conversation via SSH-launched server instances. The inter-policy launch wrapper is in `scripts/` (commit `2b3cbea`).

C.6 Section 5.2 and appendix A.1 agentic eval (`agent_bench.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench.py
```

The script runs five strategies (`recency`, `streaming_llm`, `evoke_discard`, `evoke_breadcrumb`, `evoke_kv_restore`) at each budget. For the Appendix A.1 attention row, set `EVOKE_W_ATTENTION=0.5` and `EVOKE_ATTN_LAYER=20`, which enables the `evoke_attention` strategy.

C.7 Section 5.4 workspace probe (offline pipeline, bench rows, overhead)

The offline validation and probe distillation live in the companion `j-space` repository (`https://github.com/Anyesh/j-space`); each stage writes JSON or npz under `results/` and resumes from partial output:

```
export MODEL_NAME=Qwen/Qwen2.5-7B-Instruct QUANT=8bit
python scripts/band_location.py   && python scripts/score_episodes.py
python scripts/oracle_labels.py   && python scripts/analyze_signals.py
python scripts/eviction_eval.py   # offline gate numbers
python scripts/collect_residuals.py && python scripts/fit_probe.py
```

`fit_probe.py` emits the probe artifact (`probe_qwen2.5-7b-instruct.npz`, 137 KB) and its held-out eval. The bench rows of Table 5 then need the current fork build (the probe reads residuals via `llama_get_embeddings_layer_inp`; note the probe's layer indices are HF block outputs, read at capture id $L{+}1$):

```
EVOKE_JLENS_PROBE=models/probe_qwen2.5-7b-instruct.npz \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench.py       # jlens, jlens_h2o rows
uv run python scripts/verify_layer_inp_capture.py  # capture smoke test
uv run python scripts/bench_capture_overhead.py    # decode overhead
```

C.8 Appendix A.6 keepalive workload (`agent_bench_keepalive.py`)

```
EVOKE_MODEL_PATH=$EVOKE_MODEL_PATH \
EVOKE_BUDGETS=512,1024,2048 \
  uv run python scripts/agent_bench_keepalive.py
```

The keepalive variant has each filler-file content reference the central `config.py` so the model's attention keeps coming back to the config block, which is what the attention scorer exploits.

C.9 Multi-session pool sanity check (`verify_multi_session.py`)

```
# Server must already be running (see A.4)
EVOKE_SERVER='http://localhost:8000' EVOKE_MODEL_NAME='qwen25' \
  uv run python scripts/verify_multi_session.py
```

The script drives two sessions with distinct planted codewords through interleaved chat requests and asserts that each session retrieves its own codeword, confirming state-swap correctness.

D The Fork

The llama.cpp fork is hosted at `github.com/Anyesh/llama.cpp`. The fork tracks upstream `ggml-org/llama.cpp` and adds the EVOKE-specific commits listed below.

D.1 EVOKE-specific commits in the fork

Commit	Title	What it adds
`1f37e75e7`	EVOKE: iSWA dual-cache support in `kv_block_save/load` (#41)	`llama_get_kv_cache_swa()` and an extended save/load buffer layout that covers both base and SWA sub-caches on Gemma 3 and 4
`151f1cf93`	EVOKE: multi-layer attention capture (up to 16 layers per decode)	`llama_attn_capture_set_layers(ctx, layers, n)` writes one slab per layer in a single decode
`713f202e8`	EVOKE: attention-weight capture primitive	The original single-layer side-compute path: `llama_attn_capture_set_layer`, `_set_buffer`, `_get_dims`, `_get_written`
`9b6232446`	EVOKE: attention-only `seq_rm` and `seq_add` primitives	`llama_kv_block_seq_rm` and `llama_kv_block_seq_add` reach the attention sub-cache directly via `llama_get_kv_cache`, for "no-shift" eviction modes
`a29599f4c`	EVOKE: `kv_block` primitives now cover hybrid memory and mrope models	Extends `llama_get_kv_cache` to unwrap `llama_memory_hybrid` via `get_mem_attn()` and `llama_memory_hybrid_iswa` via `get_mem_attn()->get_base()`. Removes the mrope `seq_add()` and `get_can_shift()` guards.

D.2 The two recovery primitives, verbatim

From `include/llama.h`:

```
LLAMA_API size_t llama_kv_block_save(
    struct llama_context * ctx,
            llama_seq_id   seq_id,
               llama_pos   p0,
               llama_pos   p1,
                 uint8_t * dst,
                  size_t   cap);

// Load a buffer produced by llama_kv_block_save into seq_id starting at new_p0.
LLAMA_API bool llama_kv_block_load(
    struct llama_context * ctx,
             llama_seq_id   seq_id,
           const uint8_t * src,
                  size_t   size,
               llama_pos   new_p0);
```

`save` returns the number of bytes written (or zero on error, e.g. `cap` too small). `load` returns true on success. The caller is responsible for the buffer allocation in both directions.

D.3 Build instructions

The fork builds against the upstream llama.cpp toolchain. Requirements: CMake $\geq$ 3.14, a C++17 compiler, and (for the GPU path) CUDA Toolkit 11.8 or later (we tested with 13.1).

```
cmake -B build -DLLAMA_CUDA=ON -DBUILD_SHARED_LIBS=ON
cmake --build build --config Release
```

The Python binding (`evoke/_engine_lib.py` and `evoke/llama_engine.py`) loads the shared library indicated by the `LLAMA_CPP_LIB` environment variable (a file path). It then derives `LLAMA_CPP_LIB_PATH` (a directory) for `llama-cpp-python` 0.3.x compatibility so a single loaded module backs both the Python wrapper and the EVOKE ctypes binding. Mixing two builds (the upstream wheel and the fork) causes layout-mismatch crashes; using a single fork build everywhere is the supported configuration.

D.4 Attention-capture C API surface, verbatim

The full C API for attention-weight capture, summarized in Section 4.1, added to `include/llama.h`:

```
LLAMA_API void   llama_attn_capture_set_layer (llama_context *, int32_t layer);
LLAMA_API void   llama_attn_capture_set_layers(llama_context *, const int32_t *
    layers, int32_t n);
LLAMA_API void   llama_attn_capture_set_buffer(llama_context *, float * dst,
    size_t cap_floats);
LLAMA_API void   llama_attn_capture_get_dims  (const llama_context *, int32_t *
    n_layers,
                                               int32_t * n_q, int32_t * n_h,
    int32_t * n_kv);
LLAMA_API size_t llama_attn_capture_get_written(const llama_context *);
```

The workspace probe of Section 5.4 reads the residual stream through the fork's per-layer layer-input capture, exported with C linkage in the same header (the implementations predate the export but had C++ linkage only, invisible to ctypes; registration of the capture tensor is per-architecture in `src/models/*.cpp`):

```
LLAMA_API void    llama_set_embeddings_layer_inp(llama_context *, uint32_t lid,
    bool value);
LLAMA_API float * llama_get_embeddings_layer_inp(llama_context *, uint32_t lid);
```